Fal Processor

پردازش ترتیبی فایل‌ها با وب‌سرویس SOAP، ثبت نتایج در PostgreSQL و کنترل عملیات از طریق Redis

تاریخ بررسی: ۲۰ اکتبر ۲۰۲۶ • مبتنی بر کد و تنظیمات موجود در پوشه پروژه

این برنامه چه کاری انجام می‌دهد؟

برنامه فایل‌های متنی پوشه ورودی را انتخاب می‌کند، هر فایل را با نام جدید به پوشه آماده‌سازی انتقال می‌دهد و پس از فراخوانی وب‌سرویس، منتظر فایل‌های نتیجه می‌ماند. خطوط خروجی در پایگاه داده ثبت می‌شوند و سپس فایل بعدی پردازش می‌شود.

> انتقال SFTP و اجرای پردازش روی سرور خارجی، وظیفه این برنامه نیست. ابزار یا سرویس دیگری باید فایل آماده‌شده را منتقل کند و خروجی‌ها را در پوشه نتایج قرار دهد.

چرخه پردازش

1. خواندن فهرست فایل‌های ورودی در ابتدای یک اجرای جدید.
2. انتقال و تغییر نام فایل؛ نمونه: `fal_a.txt` → `FAL_000001_fal_a.txt`.
3. انتظار اولیه برای انتقال خارجی و ارسال نام فایل به SOAP.
4. در صورت پیدا نشدن فایل، ثبت خطا و تکرار درخواست طبق تنظیمات.
5. انتظار برای خروجی‌های لازم، ثبت خطوط، بررسی نتیجه و اعلام پیشرفت.

فهرست راهنما

بخش	صفحه
معرفی و چرخه پردازش	۱
راه‌اندازی با Docker	۲
اجرای محلی و وب‌سرویس آزمایشی	۳
تنظیمات مسیرها و SOAP	۴
تنظیمات نتایج و ساختار پروژه	۵
کنترل و پایش با Redis	۶
پایگاه داده، بازیابی و عیب‌یابی	۷

دامنه بررسی: مطالعه کد، تنظیمات و فایل‌های استقرار. اتصال واقعی به سرویس‌ها و آزمون سرتاسری در این بررسی انجام نشده است.

۲. راه‌اندازی با Docker

فایل‌های استقرار پروژه از PostgreSQL 16، Python 3.12 و Redis 7 استفاده می‌کنند. برای این روش باید Docker و دستور `docker compose` در محیط شما قابل اجرا باشند. دستورهای زیر برای PowerShell نوشته شده‌اند.

آماده‌سازی پوشه‌ها

```
Set-Location 'C:\Users\Ha-dalaei\Desktop\fal_processor'
New-Item -ItemType Directory -Force -Path `
  data/source,data/staging,data/results,data/archive
```

فایل‌های ورودی را در `data/source` کپی کنید. برنامه هنگام پردازش، این فایل‌ها را از پوشه ورودی منتقل می‌کند.

تنظیم اتصال و درخواست

در `config.json` نشانی وب‌سرویس، `soap_action`، قالب XML و الگوهای نام خروجی را متناسب با سرویس واقعی تغییر دهید. مقادیر `example.com` در قالب SOAP نمونه هستند.

برای جایگزینی مقادیر Compose، فایل `env.` را کنار `docker-compose.yml` بسازید. نمونه زیر به وب‌سرویس آزمایشی روی میزبان اشاره می‌کند؛ مقدار رمز را خودتان تعیین کنید.

```
POSTGRES_DB=fal_db
POSTGRES_USER=fal_user
POSTGRES_PASSWORD=CHANGE_ME
SOAP_URL=http://host.docker.internal:8080/FalService
```

بالا آوردن و شروع پردازش

```
docker compose up -d --build
docker compose ps
docker compose logs -f app
```

پس از آماده شدن برنامه، در ترمینال دیگری دستور شروع را ارسال کنید. مقدار پیش‌فرض `app.auto_start` برابر `false` است.

```
docker compose exec redis redis-cli publish fal:control start
```

مسیر میزبان	مسیر داخل کانتینر
`./data`	`/data`
`./config.json`	`/app/config.json`، فقط خواندنی
`./logs`	`/app/logs`

پس از ویرایش خود فایل تنظیمات، `docker compose restart app` را اجرا کنید. پس از تغییر متغیرهای محیطی Compose، با `docker compose up -d` تنظیمات جدید را اعمال کنید.

اسکریپت `sql/init.sql` هنگام مقداردهی اولیه پایگاه داده خالی اجرا می‌شود. تغییر رمز در فایل محیطی، رمز پایگاه داده‌ای را که قبلاً ساخته شده است خودکار تغییر نمی‌دهد.

۳. اجرای محلی و آزمایش

اجرای برنامه با Python روی ویندوز

از پوشه پروژه شروع کنید. دستور اول فقط PostgreSQL و Redis را با Compose اجرا می‌کند. نمونه زیر فرض می‌کند اطلاعات اتصال همان مقادیر پیش‌فرض پروژه هستند؛ اگر آن‌ها را تغییر داده‌اید، این مقادیر را نیز هماهنگ کنید.

```
docker compose up -d postgres redis
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

$env:DB_HOST = 'localhost'
$env:DB_NAME = 'fal_db'
$env:DB_USER = 'fal_user'
$env:DB_PASSWORD = 'fal_password'
$env:REDIS_HOST = 'localhost'
$env:SOURCE_DIR = "$PWD/data/source"
$env:STAGING_DIR = "$PWD/data/staging"
$env:RESULTS_DIR = "$PWD/data/results"
$env:ARCHIVE_DIR = "$PWD/data/archive"
$env:SOAP_URL = 'http://localhost:8080/FalService'

.\.venv\Scripts\python.exe main.py --config config.json
```

این دستور ساخت محیط به نصب Python 3.12 نیاز دارد؛ این نسخه با Dockerfile پروژه هم‌راستا است. برنامه فایل `env.` را مستقیماً نمی‌خواند؛ در اجرای محلی متغیرها باید در محیط فرایند تنظیم شوند.

راه‌اندازی وب‌سرویس آزمایشی

در ترمینال جداگانه و از پوشه پروژه اجرا کنید. این ابزار فایل‌های آزمایشی نتیجه را در مسیر تعیین‌شده می‌سازد.

```
python tools/mock_soap_server.py --port 8080 `
  --staging-dir data/staging --results-dir data/results `
  --sftp-delay 20 --process-delay 10
```

یک فایل متنی نمونه در `data/source` قرار دهید، برنامه را اجرا کنید و فرمان `start` بخش کنترل Redis را بفرستید. برای مشاهده تلاش مجدد، مقدار `soap.initial_delay_seconds` را مثلاً ۲ و `soap.retry_interval_seconds` را ۵ قرار دهید.

وب‌سرویس آزمایشی تا سپری شدن زمان شبیه‌سازی انتقال، خطای پیدا نشدن فایل می‌دهد و سپس سه خروجی `_summary.txt`، `_detail.txt` و `_errors.txt` تولید می‌کند. فایل خطا می‌تواند خالی باشد.

> زمان‌بندی ابزار آزمایشی بر اساس `st_ctime` است؛ روی ویندوز ممکن است زمان ایجاد اولیه فایل مبنا قرار گیرد و تأخیر شبیه‌سازی‌شده دقیق نباشد. این ابزار تأییدکننده انتقال واقعی SFTP نیست. هم‌زمان فقط یک نمونه از پردازشگر را روی همین داده‌ها اجرا کنید.

۴. تنظیمات مسیرها و SOAP

تنظیمات در config.json متمرکز هستند. عبارت‌هایی مانند ${DB_HOST:-postgres} از متغیر محیطی استفاده می‌کنند و در نبود آن، مقدار پیش‌فرض را می‌خوانند.

کلید	کاربرد
paths.source_dir	پوشه فایل‌های اولیه؛ باید موجود باشد.
paths.staging_dir	محل فایل تغییرنام‌یافته برای انتقال خارجی.
paths.archive_dir / paths.results_dir	محل دریافت نتایج و مقصد بایگانی اختیاری.
source.file_glob	پیش‌فرض *.txt؛ فهرست به ترتیب نام مرتب می‌شود.
source.min_age_seconds	حداقل زمان از آخرین تغییر فایل؛ پیش‌فرض صفر.
rename.replacement / rename.pattern	تغییر نام با عبارت منظم؛ پشتیبانی از {job_id} شش‌رقمی، {date} و {datetime}.
rename.on_no_match	fail یا keep_name.
rename.on_target_exists	error یا overwrite؛ پیش‌فرض خطا.
staging.use_temp_suffix	در انتقال بین فایل‌سیستم‌ها، کپی با پسوند موقت .part انجام می‌شود.

فراخوانی وب‌سرویس

کلید زیر بخش soap	کاربرد یا مقدار فعلی
soap_action / endpoint_url	نشانی مقصد و عملیات وب‌سرویس.
envelope_template / soap_version	نسخه ۱٫۱ یا ۱٫۲ و XML درخواست؛ {{file_name}} با نام جدید جایگزین می‌شود.
verify_ssl / auth.type	احراز هویت none/basic/bearer؛ بررسی گواهی فعلاً فعال است.
timeout_seconds	مهلت شبکه هر درخواست: ۶۰ ثانیه.
initial_delay_seconds	انتظار پیش از اولین تلاش: ۶۰ ثانیه.
retry_interval_seconds	فاصله تلاش‌ها: ۱۵ ثانیه.
max_attempts / max_wait_seconds	بودجه انتظار بین تلاش‌ها: ۹۰۰ ثانیه؛ تعداد تلاش صفر یعنی بدون سقف تعداد.
success_patterns / not_found_patterns	الگوهای تشخیص پاسخ؛ در حالت پیش‌فرض موفقیت یعنی HTTP از خانواده ۲xx بدون SOAP Fault.
retry_on_transport_error / retry_on_fault	تکرار خطای ارتباط فعال و تکرار سایر Faultها غیرفعال است.
extra_params_from_db	در صورت فعال بودن، کلیدهای جدول json_config برای جای‌گذاری در قالب XML خوانده می‌شوند.

در کد فعلی، max_wait_seconds مجموع فاصله‌های انتظار است؛ زمان درخواست‌ها و انتظار اولیه را شامل نمی‌شود و سقف قطعی زمان واقعی نیست.

۵. خروجی‌ها و ساختار پروژه

فایل‌های نتیجه

در تنظیمات فعلی هر سه نوع زیر الزامی هستند؛ فقط محتوای خلاصه برای تشخیص موفقیت بررسی می‌شود. {stem} نام تغییریافته بدون پسوند است.

نوع	الگوی نام	بررسی وضعیت
summary	{stem}_summary*	بله
detail	{stem}_detail*	خیر
errors	{stem}_errors*	خیر

کلید زیر بخش results	کاربرد / مقدار فعلی
wait_timeout_seconds / poll_interval_seconds	انتظار تا ۳۶۰۰ ثانیه بر اساس شمارش فاصله‌های بررسی؛ بررسی هر ۵ ثانیه.
stable_seconds	گذشت حداقل ۲ ثانیه از آخرین تغییر فایل؛ تضمین قفل شدن یا پایان نوشتن نیست.
encoding_errors / encoding	utf-8 و جایگزینی نویسه نامعتبر با replace.
skip_blank_lines	خطوط خالی در پایگاه داده ذخیره نمی‌شوند.
parse_json_lines / delimiter	تبدیل خط به فهرست یا JSON برای ستون parsed؛ هر دو فعلاً غیرفعال‌اند.
failure_regex / success_regex	الگوی موفقیت (?i)success؛ شکست با واژه‌های fail/failed/failure تشخیص داده می‌شود و اولویت دارد.
after_import	keep نگهداری، move بایگانی یا delete حذف؛ پیش‌فرض نگهداری.
log_preview_lines / log_preview_max_chars	فقط ۵ خط نخست با سقف ۳۰۰ نویسه برای هر خط در گزارش نمایش داده می‌شود.

اجزای اصلی

فایل	مسئولیت
app/config.py / main.py	ورود برنامه، اتصال اجزا و اعتبارسنجی تنظیمات.
app/orchestrator.py	اجرای مراحل، تکرار درخواست و مدیریت وضعیت هر فایل.
app/redis_bus.py / app/controller.py	کنترل شروع و توقف، دریافت فرمان و انتشار پیام.
app/file_manager.py	انتقال، تغییر نام، یافتن خروجی‌ها و بایگانی.
app/soap_client.py	ساخت درخواست XML و دسته‌بندی پاسخ.
app/db.py / app/result_processor.py	خواندن و ارزیابی نتایج؛ ثبت داده و نقاط ادامه.
app/logging_setup.py	گزارش کنسول، فایل چرخشی و Redis.

مسیر پیش‌فرض گزارش: logs/fal_processor.log؛ سقف هر فایل ۱۰ مگابایت با ۵ نسخه پشتیبان. ثبت همه خطوط در پایگاه داده با نمایش محدود خطوط در گزارش متفاوت است.

۶. کنترل و پایش با Redis

نام کانال‌ها در `redis.channels` قابل تغییر است. پیام‌های خروجی JSON هستند؛ فرمان ورودی می‌تواند متن ساده یا JSON مانند `{"command":"pause"}` باشد.

کانال	کاربرد
`fal:control`	دریافت فرمان کاربر.
`fal:status`	وضعیت اجرا و فایل جاری.
`fal:progress`	تعداد کل، موفق، ناموفق و درصد پیشرفت.
`fal:logs`	پیام‌های گزارش برنامه.

فرمان	اثر
`start`	ادامه آخرین اجرای ناتمام یا ساخت اجرای جدید.
`pause`	مکث در نقطه کنترل بعدی و ثبت وضعیت.
`resume`	خروج از مکث؛ در حالت غیرفعال مانند شروع عمل می‌کند.
`stop`	توقف پردازش با امکان ادامه بعدی؛ خود فرایند باز می‌ماند.
`status`	انتشار مجدد آخرین وضعیت و پیشرفت موجود در حافظه.
`shutdown`	توقف و خروج از فرایند؛ سیاست restart در Compose ممکن است آن را دوباره اجرا کند.

```
docker compose exec redis redis-cli publish fal:control pause
docker compose exec redis redis-cli publish fal:control resume
docker compose exec redis redis-cli publish fal:control stop
docker compose exec redis redis-cli publish fal:control start
docker compose exec redis redis-cli psubscribe 'fal:*'
docker compose exec redis redis-cli get fal:status:last
docker compose exec redis redis-cli get fal:progress:last
```

خواندن وضعیت

`WAITING`: منتظر شروع؛ `RUNNING`: اجرای فعال؛ `PROCESSING`: پردازش؛ `WAITING_FOR_FILE`: انتظار انتقال فایل؛ `WAITING_FOR_RESULT`: انتظار خروجی؛ `PAUSED` و `STOPPED`: مکث و توقف؛ `DONE`: پایان اجرا؛ `ERROR`: خطای اجرای کلی.

```
{"run_id":1,"total":300,"processed":100,
 "done":98,"failed":2,"percent":33.33,
 "current_file":"fal_101.txt","timestamp":"..."}
```

درصد پیشرفت از مجموع فایل‌های موفق و ناموفق محاسبه می‌شود؛ رسیدن به ۱۰۰٪ به معنی موفق بودن همه فایل‌ها نیست. پیام `DONE` نیز می‌تواند شامل فایل ناموفق باشد.

فرمان‌ها در کانال Pub/Sub صف پایدار ندارند؛ هنگام قطع اتصال ممکن است دریافت نشوند. مکث و توقف، درخواست شبکه یا عملیات پایگاه داده در حال اجرا را فوراً قطع نمی‌کنند.

۷. داده‌ها، بازیابی و عیب‌یابی

جدول پیش‌فرض	محتوا
process_runs	وضعیت کلی اجرا و شمارنده‌ها.
file_jobs	نام اولیه و جدید، مرحله، تعداد تلاش و آخرین خطا.
soap_call_errors	خطاهای SOAP، وضعیت HTTP و بخشی از پاسخ.
result_records	نوع و نام خروجی، شماره خط، متن اصلی و مقدار JSON.
json_config	تنظیمات مشترک وب‌سرویس؛ در اسکریپت SQL تعریف شده است.

نام جدول‌ها زیر database.tables قابل تنظیم است. ساخت خودکار برنامه چهار جدول عملیاتی را ایجاد می‌کند؛ جدول json_config را ایجاد نمی‌کند. اگر به این جدول نیاز دارید، اسکریپت SQL یا ساختار متناظر را در پایگاه داده فراهم کنید.

```
SELECT status, count(*) FROM file_jobs
WHERE run_id = 1 GROUP BY status;

SELECT file_name, attempt, error_message
FROM soap_call_errors ORDER BY id DESC LIMIT 20;

SELECT result_type, line_no, raw_line FROM result_records
WHERE file_name = 'FAL_000001_fal_a.txt' ORDER BY id;
```

ادامه اجرا و حدود بازیابی

مراحل هر فایل به صورت pending → moved → soap_done → done/failed ثبت می‌شود. اجرای دارای وضعیت RUNNING، PAUSED، STOPPED یا ERROR قابل ادامه است. فایل‌های تازه‌وارد در اجرای جدید بعدی انتخاب می‌شوند.

اگر processing.retry_failed_on_resume=true باشد، فایل‌های ناموفقِ اجرای ناتمام دوباره صف‌بندی می‌شوند. اجرای DONE با فرمان شروع باز نمی‌شود. مقدار processing.on_job_failure تعیین می‌کند پس از شکست یک فایل، پردازش ادامه یابد یا متوقف شود.

ثبت مجدد نتایج یک فایل، رکوردهای همان کار را در یک تراکنش جایگزین می‌کند. با این حال، SOAP و انتقال فایل با ثبت پایگاه داده تراکنش مشترک ندارند؛ قطع ناگهانی بین موفقیت SOAP و ثبت مرحله می‌تواند باعث فراخوانی مجدد شود. همچنین جابه‌جایی یا حذف نتایج پیش از ثبت نهایی، بازیابی پس از قطع ناگهانی را دشوار می‌کند.

مشکلات رایج

نشانه	بررسی پیشنهادی
ماندن در WAITING	فرمان شروع، اتصال Redis، وجود فایل مطابق الگو و حداقل سن فایل.
تکرار پیدا نشدن فایل	انتقال خارجی، نام جدید و مسیر مورد انتظار وب‌سرویس.
انتظار طولانی برای نتیجه	وجود هر سه خروجی الزامی، تطبیق نام و زمان آخرین تغییر.
شکست با وجود خروجی	متن summary و الگوهای موفقیت/شکست؛ نتیجه ذخیره‌شده در پایگاه داده.
خطای اتصال یا ERROR	گزارش برنامه، میزبان و اطلاعات اتصال؛ پس از رفع مشکل فرمان شروع.

این سند بر اساس رفتار فعلی کد تهیه شده است. فایل README انگلیسی و کد اجرایی پروژه برای تهیه این راهنما تغییر نکرده‌اند.